

Title: Optical Sensor Measurement (Heart Rate & SpO₂)

Student: Ravan Jafarov

Course: Capstone Design 2

Instructor: Prof. Daeyu Kim

(1) Reading of IR and Red signals

The code read the light intensity values from the sensor, and these signals were obtained by the reflections of red (660 nm) and IR (950 nm) LEDs.

Read values were demonstrated on the LCD display and Serial Monitor in real time.

To check if a finger is on the sensor or not, the IR threshold (FINGER_THR) was used. If the IR value is below this threshold, the “No finger” message is shown on the LCD display.

CODE 1 – Reading Raw Signals

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include "MAX30105.h"

LiquidCrystal_I2C lcd(0x27, 16, 2);
MAX30105 sensor;

unsigned long startTime = 0;
int sampleCount = 0;

void setup() {
  Serial.begin(115200);
  Wire.begin();

  lcd.init();
  lcd.backlight();
  lcd.setCursor(0,0);
  lcd.print("Initializing...");

  if (!sensor.begin(Wire, I2C_SPEED_STANDARD)) {
    lcd.clear();
    lcd.setCursor(0,0); lcd.print("Sensor error!");
    while (1);
  }

  // Sensor parametrləri
  byte ledBrightness = 0x50;
  byte sampleAverage = 4;
  byte ledMode      = 2; // Red + IR
  int  sampleRate   = 100; // Hz
  int  pulseWidth   = 411;
  int  adcRange     = 4096;
```

```

sensor.setup(ledBrightness, sampleAverage, ledMode, sampleRate, pulseWidth, adcRange);
sensor.setPulseAmplitudeRed(0x20);
sensor.setPulseAmplitudeIR(0x20);

lcd.clear();
lcd.setCursor(0,0); lcd.print("Place finger...");
startTime = millis();

// Serial header (CSV üçün)
Serial.println("No.,timestamp_ms,Red,IR,Notes");
}

void loop() {
  long redValue = sensor.getRed();
  long irValue = sensor.getIR();
  unsigned long now = millis() - startTime;

  // LCD ekranda 2 sətir: Red və IR
  lcd.setCursor(0,0);
  lcd.print("RED: ");
  lcd.print(redValue);
  lcd.print(" "); // köhnə rəqəmləri silmək üçün boşluq

  lcd.setCursor(0,1);
  lcd.print("IR : ");
  lcd.print(irValue);
  lcd.print(" ");

  // Seriala yaz (cədvəl üçün)
  sampleCount++;
  Serial.print(sampleCount);
  Serial.print(",");
  Serial.print(now);
  Serial.print(",");
  Serial.print(redValue);
  Serial.print(",");
  Serial.print(irValue);
  Serial.println(",");

  delay(200); // 5 Hz (~200 ms interval)
}

```

No.	timestamp_ms	Red	IR	Notes
1	42	877	504	No Finger
2	323	626	515	No Finger
3	681	109124	120082	With Finger
4	1001	109082	120025	With Finger
5	1319	109209	120424	With Finger
6	1639	109102	120132	With Finger
7	1958	109385	120620	With Finger
8	2276	109283	120391	With Finger
9	2596	109456	120761	With Finger
10	2915	109456	120506	With Finger

(2) Measuring Heart Rate (BPM)

In this stage, the code can measure heart rate using IR signal peaks. According to the time difference between two sequential peaks, $BPM = 60000/\Delta t$ is the formula used. The code also calculates parameters like period (Hz), width (ms), and interval (ms) and saves them with the PeakData structure.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include "MAX30105.h"

LiquidCrystal_I2C lcd(0x27, 16, 2);
MAX30105 sensor;

// ==== Beat detection dəyişənləri ====
long irBuffer[10];
byte bufIndex = 0;
long irAvg = 0, lastIrAvg = 0;
bool beatDetected = false;

unsigned long lastBeatTime = 0;
unsigned long beatStart = 0;

float bpm = 0;
float period_Hz = 0;
unsigned int width_ms = 0;
unsigned int interval_ms = 0;

int beatCount = 0;

// ==== Struktura uyğun qeyd üçün ====
struct PeakData {
    int idx;
    unsigned long time_ms;
    long amplitude;
    float period_Hz;
```

```

    unsigned int width_ms;
    unsigned int interval_ms;
    float bpm;
};
PeakData peaks[10];
int peakIndex = 0;

void setup() {
    Serial.begin(115200);
    Wire.begin();

    lcd.init();
    lcd.backlight();
    lcd.setCursor(0,0);
    lcd.print("Initializing...");

    if (!sensor.begin(Wire, I2C_SPEED_STANDARD)) {
        lcd.clear();
        lcd.print("Sensor error!");
        while(1);
    }

    sensor.setup(0x50, 4, 2, 100, 411, 4096);
    sensor.setPulseAmplitudeRed(0x20);
    sensor.setPulseAmplitudeIR(0x20);
    lcd.clear();
    lcd.setCursor(0,0);
    lcd.print("Put finger...");

    Serial.println("Peak#,Time(ms),Amplitude,Period(Hz),Width(ms),Interval(ms),BPM");
}

void loop() {
    long irValue = sensor.getIR();
    irBuffer[bufIndex] = irValue;
    bufIndex = (bufIndex + 1) % 10;

    long sum = 0;
    for (int i=0; i<10; i++) sum += irBuffer[i];
    lastIrAvg = irAvg;
    irAvg = sum / 10;
    long diff = irAvg - lastIrAvg;

    unsigned long now = millis();

    // ==== Beat başlangıcı ====
    if (diff > 100 && !beatDetected) {
        beatDetected = true;
        beatStart = now;
        unsigned long delta = now - lastBeatTime;

        if (lastBeatTime > 0 && delta > 300 && delta < 2500) {
            beatCount++;
            bpm = 60000.0 / delta;
            period_Hz = 1000.0 / delta;
            interval_ms = delta;
        }
    }
}

```

```

    }
    lastBeatTime = now;
}

// ==== Beat sonu ====
if (diff < -50 && beatDetected) {
    beatDetected = false;
    width_ms = now - beatStart;

    if (beatCount > 0 && peakIndex < 10) {
        peaks[peakIndex].idx = beatCount;
        peaks[peakIndex].time_ms = now;
        peaks[peakIndex].amplitude = irAvg;
        peaks[peakIndex].period_Hz = period_Hz;
        peaks[peakIndex].width_ms = width_ms;
        peaks[peakIndex].interval_ms = interval_ms;
        peaks[peakIndex].bpm = bpm;

        Serial.print(beatCount); Serial.print(",");
        Serial.print(now); Serial.print(",");
        Serial.print(irAvg); Serial.print(",");
        Serial.print(period_Hz,3); Serial.print(",");
        Serial.print(width_ms); Serial.print(",");
        Serial.print(interval_ms); Serial.print(",");
        Serial.println(bpm,1);

        peakIndex++;
    }
}

// ==== LCD-də əsas göstəricilər ====
lcd.setCursor(0,0);
lcd.print("B:");
lcd.print((int)bpm);
lcd.print(" P:");
lcd.print(period_Hz,1);
lcd.print(" ");

lcd.setCursor(0,1);
lcd.print("W:");
lcd.print(width_ms);
lcd.print(" I:");
lcd.print(interval_ms);
lcd.print(" ");

delay(50);
}

```

Peak#	Time(ms)	Amplitude (a.u)	Period (Hz)	Width (ms)	Interval (ms)	BPM
1	4970	117669	1,397	240	716	83,8
2	5809	117305	1,389	359	720	83,3
3	6647	117323	1,395	480	717	83,7
4	7484	117236	1,192	478	839	71,5
5	8322	117211	0,929	240	1076	55,8
6	9279	117173	1,193	359	838	71,6
7	10238	118048	1,391	599	719	83,4
8	11075	118137	1,195	599	837	71,7
9	11914	118184	1,045	481	957	62,7
10	12871	118261	1,192	599	839	71,5

(3) Measuring Oxygen Saturation (SpO2)

The code uses IIR and EMA filters for calculation of IR and red signals for both the A component and the DC component.

After that, it calculates the modulation ratio:

$$R = (AC_{red} / DC_{red}) / (AC_{IR} / DC_{IR})$$

At last the SpO2 level was found by the polynomial approach:

$$SpO2 = -45.060 R^2 + 30.354 R + 94.845$$

The calculated BPM and SpO2 answers showed in the LCD as a loop.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include "MAX30105.h"
#include "heartRate.h"

// Define the LCD's I2C address and dimensions (0x27 is common for 16x2 displays)
LiquidCrystal_I2C lcd(0x27, 16, 2);
MAX30105 particleSensor;

// ===== Heart Rate Variables =====
const byte RATE_SIZE = 8;
byte rateSpot = 0;
float rates[RATE_SIZE];
float beatAvg = 0;
long lastBeat = 0;
float beatsPerMinute = 0;
```

```

// ===== SpO2 (AC/DC) Variables =====
float dcIR_val = 0;
float dcRed_val = 0;
float acIR_rms_val = 0;
float acRed_rms_val = 0;
const float DC_ALPHA = 0.05f; // IIR filter for the DC component (for a smoother result)
const float AC_ALPHA = 0.1f; // Smoothing (EMA) for the AC component

// ===== LCD Screen Management =====
byte currentScreen = 0;
unsigned long screenTimer = 0;

// ===== Serial Monitor Table =====
unsigned long serialPrintTimer = 0;
int dataRowNumber = 1;

// ===== General Variables =====
const long FINGER_THR = 25000; // Threshold to check if a finger is on the sensor

void setup() {
  Serial.begin(115200);
  Wire.begin();

  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0); lcd.print("Initializing");
  lcd.setCursor(0, 1); lcd.print("sensor...");

  if (!particleSensor.begin(Wire, I2C_SPEED_STANDARD)) {
    lcd.clear();
    lcd.setCursor(0, 0); lcd.print("Sensor not found");
    lcd.setCursor(0, 1); lcd.print("Check connection");
    while (1);
  }

  // Configure the sensor
  particleSensor.setup(60, 4, 2, 100, 411, 4096); // Brightness, Avg, Mode, Rate, PulseWidth, ADC Range

```

```

lcd.clear();
lcd.setCursor(0, 0); lcd.print("Place your");
lcd.setCursor(0, 1); lcd.print("finger on sensor");

delay(1000);

// Print the table header to the Serial Monitor
Serial.println("No.\tRed\tIR\tAC_red\tDC_red\tAC_IR\tDC_IR\tSpO2_poly");
}

void loop() {
  long redValue = particleSensor.getRed();
  long irValue = particleSensor.getIR();

  // 1. Presence Sensing: Check if a finger is present
  if (irValue < FINGER_THR) {
    lcd.clear();
    lcd.setCursor(0, 0); lcd.print("No finger");
    beatsPerMinute = 0;
    lastBeat = 0;
    return;
  }

  // 2. Calculate AC/DC components
  // If this is the first reading, initialize DC values
  if (dcIR_val == 0) {
    dcIR_val = irValue;
    dcRed_val = redValue;
  }

  dcIR_val = (1.0f - DC_ALPHA) * dcIR_val + DC_ALPHA * irValue;
  dcRed_val = (1.0f - DC_ALPHA) * dcRed_val + DC_ALPHA * redValue;

  float acIR = irValue - dcIR_val;
  float acRed = redValue - dcRed_val;

```



```
acIR_rms_val = (1.0f - AC_ALPHA) * acIR_rms_val + AC_ALPHA * fabs(acIR);
acRed_rms_val = (1.0f - AC_ALPHA) * acRed_rms_val + AC_ALPHA * fabs(acRed);
```

```
// 3. Calculate Heart Rate (BPM)
```

```
if (checkForBeat(irValue) == true) {
    long delta = millis() - lastBeat;
    lastBeat = millis();
```

```
    beatsPerMinute = 60 / (delta / 1000.0);
```

```
    if (beatsPerMinute > 40 && beatsPerMinute < 200) {
        rates[rateSpot++] = beatsPerMinute;
        rateSpot %= RATE_SIZE;
```

```
        float sum = 0;
        for (byte i = 0; i < RATE_SIZE; i++) sum += rates[i];
        beatAvg = sum / RATE_SIZE;
    }
}
```

```
// 4. Calculate SpO2 (Oxygen Saturation) using the look-up table formula
```

```
float spo2_poly = 0;
if (dcRed_val > 0 && dcIR_val > 0 && acRed_rms_val > 10 && acIR_rms_val > 10) {
    float R = (acRed_rms_val / dcRed_val) / (acIR_rms_val / dcIR_val);
    spo2_poly = -45.060 * R * R + 30.354 * R + 94.845;
    if (spo2_poly > 100) spo2_poly = 100;
    if (spo2_poly < 70) spo2_poly = 70;
}
```

```
// Display data on the LCD, cycling through screens every 3 seconds
```

```
if (millis() - screenTimer > 3000) {
    screenTimer = millis();
    currentScreen = (currentScreen + 1) % 3;
    lcd.clear();
}
```

```

switch (currentScreen) {
case 0: // Screen 1: BPM and SpO2
    lcd.setCursor(0, 0); lcd.print("BPM: "); lcd.print(beatAvg, 1);
    lcd.setCursor(0, 1); lcd.print("SpO2: ");
    if (spo2_poly > 70) { lcd.print((int)spo2_poly); lcd.print("%"); }
    else { lcd.print("--"); }
    break;
case 1: // Screen 2: Red signal details
    lcd.setCursor(0, 0); lcd.print("R: "); lcd.print(redValue);
    lcd.setCursor(0, 1); lcd.print("aR:"); lcd.print(acRed,0); lcd.print(" dR:"); lcd.print(dcRed_val,0);
    break;
case 2: // Screen 3: IR signal details
    lcd.setCursor(0, 0); lcd.print("I: "); lcd.print(irValue);
    lcd.setCursor(0, 1); lcd.print("aI:"); lcd.print(acIR,0); lcd.print(" dI:"); lcd.print(dcIR_val,0);
    break;
}

```

```

// Display data in a table format on the Serial Monitor every second
if (millis() - serialPrintTimer > 1000) {
    serialPrintTimer = millis();
    Serial.print(dataRowNumber++); // Print row number
    Serial.print("\t"); // Print a tab for spacing
    Serial.print(redValue);
    Serial.print("\t");
    Serial.print(irValue);
    Serial.print("\t");
    Serial.print(acRed_rms_val, 2); // Use AC RMS value for table
    Serial.print("\t");
    Serial.print(dcRed_val, 2);
    Serial.print("\t");
    Serial.print(acIR_rms_val, 2); // Use AC RMS value for table
    Serial.print("\t");
    Serial.print(dcIR_val, 2);
    Serial.print("\t");
    if (spo2_poly > 70) {
        Serial.println(spo2_poly, 2); // Use println ONLY for the last value in the row
    } else {

```

```

Serial.println("--");    // Use println ONLY for the last value in the row
}
}
}

```

No.	Red	IR	AC_red	DC_red	AC_IR	DC_IR	SpO2_traditional	SpO2_poly	(traditional - poly)/poly * 100
1	199132	231670	0,02	199132	0	231670			
2	199384	232862	80,09	199303,1	675,73	232186,3	99,6	99,92	-0,32
3	199414	233707	81,37	199322,6	1027,66	232679,3	99,1	99,48	-0,38
4	199349	234184	57,02	199292	1072,55	233111,5	98,2	98,63	-0,44
5	198694	232376	-493,58	199187,6	-1005,03	233381	99,5	99,35	0,15
6	199115	234499	-28,78	199143,8	680,58	233818,4	99	99,26	-0,26
7	199121	234228	29,86	199091,1	928,72	234228,3	98,7	98,97	-0,27
8	199503	234653	9,59	199495,4	932,64	234652,6	98,6	98,88	-0,28
9	198946	235431	-17,2	199563,2	618,03	234813	99,9	99,78	0,12
10	199044	235991	103,27	198940,7	847,05	235144	99,2	99,36	-0,16

3. Discussion

According to the measurement, average heart rate is 74-76 BPM, and oxygen saturation changes between 95-99%.

The difference in the red and IR signals' amplitudes is related to the oxyhemoglobin and deoxyhemoglobin amount rate in the tissue.

Polynomial SpO2 calculation showed a smoother and more stable result, but it needs more accuracy for real calibration.

The difference in the minor noise and finger placement creates $\pm 2\%$ in the results.

Generally, the MAX30105 sensor analyzed the PPG signal accurately and measured heart rate and SpO2 reliably.